

hibernate-labs-v3

하이버네이트 테코톡 실습 문서

대상 발표: "하이버네이트는 어떻게 JPQL을 SQL로 번역할까 - SQM에서 SQL AST까지, Hibernate의 Query Compiler" **실험 도메인:** roomescape (spring-roomescape-waiting)

1. 이 문서가 하려는 일

발표에서 단언하는 문장에는 직접 찍어본 근거가 하나씩 붙어 있어야 한다. SQM은 한국어 자료가 사실상 없고 영어권 자료도 얇다. 인용할 블로그가 없다는 뜻이고, 동시에 틀려도 아무도 미리 잡아주지 않는다는 뜻이다. 그래서 이 문서의 실습은 전부 "대본의 어느 문장을 증명하는가"에서 출발한다.

실습과 대본의 대응

이 표를 먼저 보고 시작한다. 어떤 실습을 왜 하는지가 여기에 다 들어 있다.

대본에서 하는 말	위치	증명하는 실습
"자바 코드는 한 글자도 안 바뀌었는데 SQL이 다릅니다"	도입 (1파트)	실습 2
"입력 방식은 셋인데 모이는 곳은 하나입니다"	2파트	실습 1
"SQM에는 구문이 아니라 의미가 들어 있습니다"	3파트	실습 3
"갈라지는 건 마지막 한 걸음입니다"	4-3	실습 2
"SQM에서 이건 노드 하나입니다"	5파트	실습 4
"번역하면 결과물이 두 개 나옵니다"	6파트	실습 5

실습 1부터 5까지가 발표를 구성한다. 이 다섯 개가 없으면 해당 부분의 슬라이드를 만들 수 없다.

실습 6부터 9는 성격이 다르다. 슬라이드로 가지 않고, 질문이 들어왔을 때 답하기 위해 한다.

실습	대비하는 질문
실습 6 (TYPE 연산자)	"상속 말고 다른 예는 없나요" / 5파트가 시간을 넘길 때의 대타
실습 7 (N+1 정량)	"그래서 N+1은 어떻게 되나요"
실습 8 (쿼리 플랜 캐시)	"매번 파싱하면 느리지 않나요"
실습 9 (ANTLR 문법 파일)	"정말 문법 파일이 있나요"

슬라이드에 올릴 것과 올리지 않을 것

실습 6부터 9의 결과물은 **본문 슬라이드에 넣지 않는다**. 맨 뒤에 백업 슬라이드로 붙여두고 질문이 나올 때만 꺼낸다.

이 구분을 앞에 못 박아두는 이유가 있다. 실습을 하다 보면 결과가 예쁘게 나오는 순간이 온다. 특히 실습 7의 N+1 측정 표가 그렇다. 아까워서 본문에 넣고 싶어지는데, 넣는 순간 40초짜리 파트가 3분이 되고 발표가 통째로 N+1 발표로 되돌아간다. 그때 이 절을 다시 읽는다.

태그 표기

각 항목에 붙는 태그는 그 답을 어디서 구해야 하는지를 뜻한다.

- **[내부]** 대본과 지금까지 정리한 내용만으로 답할 수 있다.
- **[탐색]** 하이버네이트 소스나 공식 문서를 직접 열어봐야 한다. 검색으로는 안 나온다.
- **[사고]** 어디에도 적혀 있지 않다. 직접 구성해야 한다.

진행 순서

1. 실습 0으로 환경을 만든다. 상속 계층까지 한 번에 세팅한다.
2. 실습 1과 2를 돌린다. 여기까지 하면 발표의 뼈대가 선다.
3. 과제 4-1에 답한다.
4. 실습 3과 4를 돌린다. 발표의 본체가 여기서 나온다.
5. 과제 4-2에 답한다.
6. 실습 5를 돌린다.
7. 남은 실습과 과제 4-3을 처리한다.

시간이 부족하면 **실습 1, 2, 4**까지만 해도 발표는 성립한다. 반대로 실습 7만 있으면 성립하지 않는다.

2. 실험 환경

미션 코드는 건드리지 않는다

지금 roomescape는 JdbcTemplate 기반이다. 이 실험을 하겠다고 미션 브랜치를 JPA로 갈아엎지 않는다. 별도 브랜치(spike/hibernate-lowering)나 별도 Gradle 모듈을 만들어서 거기서 논다.

원본 스키마를 그대로 가져오지도 않는다. 재사용하는 것은 스키마가 아니라 도메인 구조다. DDL은 하이버네이트가 생성하게 둔다(ddl-auto=create). 목표는 돌아가는 서비스가 아니라 관측할 수 있는 최소 실험대다.

H2를 쓰지 않는다

이 실험의 절반은 "DB가 다르면 SQL이 다르다"를 보이는 것이다. 그걸 H2 하나로 하면 실험 설계 자체가 성립하지 않는다. Testcontainers로 MySQL과 PostgreSQL을 각각 띄운다.

매핑할 때 주의할 것

원본 스키마를 그대로 옮기면 관측 기회를 잃는 지점이 몇 군데 있다.

reservation.date 와 reservation_time.start_at 은 원본에서 VARCHAR(255) 다. 그대로 두면 Dialect마다 날짜를 어떻게 다루는지 볼 기회가 사라진다. LocalDate 와 LocalDateTime 으로 매핑한다.

member.role 의 CHECK 제약은 문제가 없다. @Enumerated(STRING) 으로 둔다.

가장 중요한 건 ID 생성 전략이다. 원본은 전 테이블이 AUTO_INCREMENT 인데, 이걸 IDENTITY 로 매핑하면 TABLE_PER_CLASS 상속에서 애플리케이션 부팅이 실패한다. UNION 구조에서는 각 자식 테이블이 자기 auto increment를 독립적으로 돌리기 때문에 ID가 겹친다. 그래서 처음부터 전 테이블을 SEQUENCE 로 통일한다.

여기서 미루면 나중에 고생한다. 상속 실습은 발표의 킬링 포인트라 반드시 해야 하는데, 나중에 ID 전략을 갈아끼우면 그 앞에서 찍어둔 SQL 로그를 전부 다시 찍어야 한다.

참고로 MySQL은 SEQUENCE를 테이블로 흉내내기 때문에, 상속 실습(실습 4)만큼은 PostgreSQL로 돌리는 편이 결과가 깔끔하다. 실습 2는 두 DB가 다 필요하므로 컨테이너는 둘 다 띄워둔다.

엔티티를 두 벌 만든다

여기가 헛갈리기 쉬운 지점이다. 이 실험대에는 **서로 다른 모델이 두 벌** 필요하다.

- **평면 모델**: Reservation 과 ReservationWait 이 별개 엔티티다. 지금 roomescape 구조와 같다.
- **상속 모델**: SlotClaim 을 부모로 두고 예약과 대기가 자식이다. 실습 4에서 매핑 전략을 바꿔가며 관측할 대상이다.

둘을 하나로 합칠 수 없다. 실습 4에서 @Inheritance 전략을 바꾸면 그 계층에 속한 엔티티의 SQL이 전부 달라지는데, 실습 1, 2, 3, 5는 전략과 무관하게 안정적인 SQL이 나와야 비교가 된다. 상속 계층에 없어도면 앞선 실습의 기준선이 실습 4를 돌릴 때마다 흔들린다.

그래서 참조 대상(Member, Theme, ReservationTime, Store)은 공유하고, 예약과 대기 쪽만 두 벌로 나눈다.

실습	쓰는 모델
1, 2, 3, 5, 7, 8	평면 모델
4, 6	상속 모델

데이터 부담은 한쪽에만 있다. 평면 모델은 예약 128건 이상이 필요하지만, 상속 모델은 SQL 모양만 보면 되므로 몇 건이면 충분하다.

JPA에서 엔티티 이름은 영속성 단위 안에서 유일해야 하므로 상속 쪽 클래스에는 접두사를 붙인다. 발표 슬라이드에는 Reservation 과 Waiting 으로 단순화해서 보여줘도 되지만, 스파이크 코드에서는 이름이 겹치면 안 된다.

공통 부모

시퀀스를 하나로 통일하기 위한 장치다. 전략적인 이유가 있으니 그냥 넘기지 않는다.

```

@MappedSuperclass
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "spike_gen")
    @SequenceGenerator(name = "spike_gen", sequenceName = "spike_seq",
allocationSize = 50)
    private Long id;

    public Long getId() {
        return id;
    }
}

```

`@SequenceGenerator` 를 여기 한 번만 선언하고 모든 엔티티가 이걸 상속받게 한다.

TABLE_PER_CLASS 때문이다. 하이버네이트 6는 아무것도 지정하지 않으면 엔티티마다 시퀀스를 따로 만드는데, 그러면 `TABLE_PER_CLASS` 에서 두 자식 테이블이 각자 1부터 번호를 매기면서 ID가 겹친다. UNION으로 합쳤을 때 같은 ID가 두 개 나오는 상황이 된다.

`allocationSize = 50` 은 시퀀스 호출 횟수를 줄이지만, 데이터를 넣는 동안 `nextval` 호출이 로그에 섞여 나온다. 실습 7에서 쿼리 수를 셀 때는 데이터 세팅을 끝낸 뒤에 `stats.clear()` 를 부르고 측정을 시작한다.

공유하는 참조 엔티티

```

@Entity
@Table(name = "store")
public class Store extends BaseEntity {
    private String name;

    protected Store() {}
}

@Entity
@Table(name = "member")
public class Member extends BaseEntity {
    private String name;

    @Enumerated(EnumType.STRING)
    private Role role;

    @ManyToOne(fetch = FetchType.LAZY)
    private Store store;    // 매니저인 경우만 채운다. nullable

    protected Member() {}
}

public enum Role { USER, ADMIN }

@Entity
@Table(name = "theme")
public class Theme extends BaseEntity {
    private String name;
    private String description;

    protected Theme() {}
}

@Entity
@Table(name = "reservation_time")
public class ReservationTime extends BaseEntity {
    private LocalDateTime startAt;    // 원본은 VARCHAR였다. 반드시 LocalDateTime으로

    protected ReservationTime() {}
}

```

평면 모델 (실습 1, 2, 3, 5, 7, 8)

```

@Entity
@Table(name = "reservation")
public class Reservation extends BaseEntity {

    private LocalDate date;    // 원본은 VARCHAR였다. 반드시 LocalDate로

    @ManyToOne(fetch = FetchType.LAZY)
    private Member member;

    @ManyToOne(fetch = FetchType.LAZY)
    private ReservationTime time;

    @ManyToOne(fetch = FetchType.LAZY)    // 실습 7에서 이 줄을 EAGER로 토글한다
    private Theme theme;

    @ManyToOne(fetch = FetchType.LAZY)
    private Store store;

    @OneToMany(mappedBy = "reservation")
    private List<ReservationWait> waitings = new ArrayList<>();

    protected Reservation() {}
}

@Entity
@Table(name = "reservation_wait")
public class ReservationWait extends BaseEntity {

    @ManyToOne(fetch = FetchType.LAZY)
    private Reservation reservation;

    @ManyToOne(fetch = FetchType.LAZY)
    private Member member;

    private LocalDateTime createdAt;

    protected ReservationWait() {}
}

```

컬렉션은 `Reservation.waitings` 하나만 둔다. 실습 5에서 `JOIN FETCH r.waitings` 로 "평평한 행이 객체 그래프로 조립되는" 장면을 보려면 컬렉션이 하나는 있어야 한다. 반대로 컬렉

선이 둘 이상이면 DomainResult 덤프가 읽을 수 없을 만큼 길어진다.

ReservationWait 이 Reservation 을 참조하기 때문에, 대기 목록을 조회하면 연관을 두 단계 타고 들어가게 된다. 실습 7의 2단계 깊이 관측에서 이 구조를 쓴다.

상속 모델 (실습 4, 6)

```
@Entity
@Table(name = "slot_claim")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE) // 실습 4에서 이 한 줄만 바꾼다
@DiscriminatorColumn(name = "claim_type")
public abstract class SlotClaim extends BaseEntity {

    private LocalDate date;

    @ManyToOne(fetch = FetchType.LAZY) private Member member;
    @ManyToOne(fetch = FetchType.LAZY) private Theme theme;
    @ManyToOne(fetch = FetchType.LAZY) private ReservationTime time;
    @ManyToOne(fetch = FetchType.LAZY) private Store store;

    protected SlotClaim() {}
}

@Entity
@Table(name = "slot_reservation")
@DiscriminatorValue("RESERVATION")
public class SlotReservation extends SlotClaim {
    protected SlotReservation() {}
}

@Entity
@Table(name = "slot_waiting")
@DiscriminatorValue("WAITING")
public class SlotWaiting extends SlotClaim {

    private LocalDateTime createdAt;

    protected SlotWaiting() {}
}
```


실습 4에서는 `@Inheritance` 의 `strategy` 만 바꾸고 나머지는 그대로 둔다. 자식 클래스의 `@Table` 은 `SINGLE_TABLE` 에서 무시되고 `JOINED` 와 `TABLE_PER_CLASS` 에서 쓰인다. `@DiscriminatorColumn` 도 `SINGLE_TABLE` 에서만 의미가 있다. **두 어노테이션을 그대로 붙여 둔 채로 전략만 갈아끼우는 게 요령이다.** 전략마다 어노테이션까지 손대면 무엇 때문에 SQL 이 바뀌었는지 헛갈린다.

`SlotWaiting.createdAt` 은 `SINGLE_TABLE` 에서 nullable 컬럼이 된다. 자식에만 있는 필드라 당연한 결과인데, 이것 자체가 `SINGLE_TABLE` 의 대가를 보여주는 재료다. 과제 Q9에서 쓴다.

매핑할 때 걸리는 함정

`@ManyToOne` 의 기본값은 **EAGER**다. 명시적으로 `fetch = FetchType.LAZY` 를 적지 않으면 전부 즉시 로딩이 되고, 실습 1과 2에서 찍히는 SQL에 원치 않는 JOIN이 섞여 들어온다. 위 코드처럼 전부 명시한다.

실습 7에서 EAGER 조건을 볼 때는 애노테이션을 고치고 애플리케이션을 다시 띄워야 한다. 조건을 왔다 갔다 하면 시간이 많이 드니, LAZY 조건 측정을 전부 끝낸 뒤에 EAGER로 바꾸는 식으로 묶어서 돌린다.

`toString()` 에 **연관 필드를 넣지 않는다.** Lombok의 `@Data` 나 `@ToString` 도 붙이지 않는다. 프록시를 건드리는 순간 초기화되면서 쿼리가 나가고, 실습 7의 측정이 통째로 오염된다. 같은 이유로 실습 7은 IDE 디버거 변수 창을 열어서 확인하지 말고 로그로만 확인한다. 변수 창을 펼치는 것만으로도 프록시가 초기화된다.

`equals` 와 `hashCode` 는 **건드리지 않는다.** 이 실험대에는 도메인 로직이 없어서 기본 구현으로 충분하다. ID 기반 equals를 어설프게 넣으면 영속성 컨텍스트가 같은 엔티티를 어떻게 취급하는지 판단하기가 더 어려워진다.

기본 생성자는 **protected** 로 둔다. 하이버네이트가 프록시를 만들려면 인자 없는 생성자에 접근할 수 있어야 하는데, `private` 이면 프록시 생성이 막힌다.

로깅과 의존성

```
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.orm.query.hql=TRACE
logging.level.org.hibernate.orm.query.sqm.ast=DEBUG
logging.level.org.hibernate.orm.sql.ast.create=DEBUG
logging.level.org.hibernate.orm.results.graph.AST=DEBUG
spring.jpa.properties.hibernate.generate_statistics=true
spring.jpa.hibernate.ddl-auto=create
```

각 줄이 무엇을 찍어주는지 알고 커야 로그를 읽을 수 있다.

카테고리	찍히는 것	쓰는 실습
org.hibernate.SQL	최종 SQL 문자열	전부
org.hibernate.orm.query.hql	HQL 파싱 과정	1
org.hibernate.orm.query.sqm.ast	SQM 트리 덤프	1, 2, 4
org.hibernate.orm.sql.ast.create	SQL AST 생성 과정	2, 5
org.hibernate.orm.results.graph.AST	DomainResult 그래프 덤프	5

마지막 줄이 실습 5의 핵심이다. `DomainResultGraphPrinter` 가 이 카테고리로 찍는다.

주의할 게 두 가지 있다.

대소문자를 지킨다. 카테고리 이름 끝의 `AST` 는 대문자다. 스프링 부트의 `logging.level.*` 은 프로퍼티 키를 완화된 규칙으로 바인딩하기 때문에 대문자 키가 그대로 전달되지 않는 경우가 있다. 위 설정을 넣었는데 아무것도 안 찍히면 이게 원인일 가능성이 높다. 그때는 `logback-spring.xml` 에 로거를 직접 선언한다.

```
<logger name="org.hibernate.orm.results.graph.AST" level="DEBUG"/>
```

켜는 시점이 늦으면 안 된다. 하이버네이트의 덤프 출력 클래스들은 로거가 `DEBUG`인지를 클래스가 로드될 때 한 번 확인해서 상수로 들고 있는 경우가 있다. 애플리케이션이 뜬 뒤에 로그 레벨을 바꾸면 덤프가 안 나올 수 있으니, 설정 파일에 미리 넣고 시작한다.

```
implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
testImplementation 'org.springframework.boot:spring-boot-testcontainers'
testImplementation 'org.testcontainers:testcontainers-junit-jupiter'
testImplementation 'org.testcontainers:testcontainers-mysql'
testImplementation 'org.testcontainers:testcontainers-postgresql'
testImplementation 'net.ttddyy:datasource-proxy:1.10.1'
```

좌표를 틀리기 쉬운 지점이 두 군데 있다.

Testcontainers 2.0에서 모든 모듈 아티팩트 이름 앞에 testcontainers- 가 붙었다. 예전에 쓰던 `org.testcontainers:mysql` 은 이제 `org.testcontainers:testcontainers-mysql` 이다. 스프링 부트 4의 BOM은 새 좌표만 관리하므로, 옛 좌표를 쓰면 버전이 안 잡혀서 해석 자체가 실패한다.

datasource-proxy 는 스프링 부트 BOM 관리 대상이 아니다. 버전을 직접 적어야 한다.

증상이 오해를 부르는 조합이라 미리 적어둔다. 좌표 하나가 틀리면 테스트 클래스패스가 통째로 안 만들어지고, 그러면 IDE에서 **애노테이션 자동완성이 아예 안 뜬다**. 애노테이션 이름을 틀린 것처럼 보이지만 실제로는 의존성 문제다. 이럴 때는 먼저 이걸 돌려서 `FAILED` 가 있는지 본다.

```
./gradlew dependencies --configuration testRuntimeClasspath
```

애노테이션과 import에서 틀리기 쉬운 것

`@Testcontainers` 다. `@TestContainers` 가 아니다. `Testcontainers`가 제품명 한 단어라서 가운데 c가 소문자다. IDE에서 `TestC` 까지 치면 후보가 안 나온다.

애노테이션이 어느 라이브러리에서 오는지도 구분해둔다. 하나가 안 뜬다면 그 라이브러리가 안 붙은 것이다.

애노테이션	패키지	의존성
<code>@Testcontainers</code> , <code>@Container</code>	<code>org.testcontainers.junit.jupiter</code>	<code>testcontainers-junit-jupiter</code>
<code>@ServiceConnection</code> , <code>@ImportTestcontainers</code>	<code>org.springframework.boot.testcontainers.*</code>	<code>spring-boot-testcontainers</code>

컨테이너 클래스의 import 경로는 2.0에서 옮겨졌다. 새 위치는

`org.testcontainers.mysql.MySQLContainer` 와

`org.testcontainers.postgresql.PostgreSQLContainer` 다. 옛 경로

(`org.testcontainers.containers.*`)도 아직 jar에 남아 있어서 자동 import가 그쪽을 집어올 수 있으니, 컴파일이 되더라도 어느 쪽을 잡았는지 한 번 확인한다.

`@Testcontainers` 만 붙이면 컨테이너가 뜨지 않는다. 이 애노테이션은 `@Container` 가 붙은 필드를 찾아서 생명주기를 관리하는 확장일 뿐이다. `@Container` 필드가 없으면 아무 컨테이너도 안 뜨고, `application.properties` 의 `datasource` 설정으로 붙으려 하다가 실패한다. 아래 골격처럼 `@Container` 와 `@ServiceConnection` 을 필드에 함께 붙여야 한다.

실습을 어디서 돌리나

`main` 이 아니라 테스트에서 돌린다. 세 가지 이유가 있다.

첫째, `Testcontainers`를 띄우는 자리가 테스트다. 둘째, 실습 2는 같은 쿼리를 MySQL과 PostgreSQL에서 각각 돌려야 하는데, 테스트 클래스를 나누면 그게 그대로 조건 분리가 된다. 셋째, 실습 7에서 `em.clear()` 와 트랜잭션 경계를 손으로 제어해야 하는데 테스트가 그걸 하기 가장 쉽다.

`main` 에서 돌리면 앱 부팅 로그에 SQM 덤프가 파묻혀서 캡처하기도 나쁘다.

기본 골격

```
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import
org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.springframework.transaction.annotation.Transactional;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;
import org.testcontainers.mysql.MySQLContainer;

@SpringBootTest
@Testcontainers
@Transactional
class MySqlLoweringTest {

    @Container
    @ServiceConnection
    static MySQLContainer<?> db = new MySQLContainer<>("mysql:8.4");

    @PersistenceContext
    EntityManager em;

    @Test
    void 페이지징_쿼리는_어떤_SQL로_나가는가() {
        System.out.println("=== 여기부터 ===");

        em.createQuery("SELECT r FROM Reservation r WHERE r.theme.name = :name",
                        Reservation.class)
            .setParameter("name", "공포")
            .setMaxResults(10)
            .getResultList();

        System.out.println("=== 여기까지 ===");
    }
}
```

구분선을 찍는 이유는 실용적이다. 로그가 길어서 어디부터가 이 쿼리인지 찾기 어렵다. 테스트 하나에 쿼리 하나만 넣고 앞뒤를 표시해두면 캡처할 구간이 바로 보인다.

@Transactional 이 붙어 있으면 테스트가 끝날 때 롤백되므로 조건 사이에 데이터가 오염되지 않는다. 그리고 지연 로딩을 관측하려면 세션이 열려 있어야 하는데, 이 애노테이션이 그 역할도 한다. 실습 7에서 이게 없으면 프록시를 건드리는 순간 예외가 난다.

실습 2에서 DB 두 개 돌리기

같은 테스트를 컨테이너만 바꿔 두 번 돌린다. 부모 클래스에 쿼리를 두고 자식에서 컨테이너만 갈아끼우면 조건이 하나만 달라진다는 게 코드로 보장된다.

```
abstract class LoweringTestBase {

    @PersistenceContext
    EntityManager em;

    @Test
    void 페이징_쿼리는_어떤_SQL로_나가는가() {
        // 위와 같은 내용
    }
}

@SpringBootTest @Testcontainers @Transactional
class MySqlLoweringTest extends LoweringTestBase {
    @Container @ServiceConnection
    static MySQLContainer<?> db = new MySQLContainer<>("mysql:8.4");
}

@SpringBootTest @Testcontainers @Transactional
class PostgresLoweringTest extends LoweringTestBase {
    @Container @ServiceConnection
    static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16");
}
```

PostgreSQLContainer 의 import는 org.testcontainers.postgresql.PostgreSQLContainer 다.

알아두면 로그 읽기가 쉬워지는 것

SQM 덤프는 getResultList() 가 아니라 createQuery() 에서 찍힌다. HQL 파싱과 SQM 생성이 그 시점에 일어나기 때문이다. 즉 로그에서 createQuery 이후에 SQM 덤프가 나오고, 그다음에 SQL AST 덤프가 나오고, 마지막으로 org.hibernate.SQL 이 나온다. 이 순서 자체가 발표에서 설명하는 파이프라인이다.

덕분에 실습 3(오타 실험)은 `getResultList()` 를 부를 필요조차 없다. `createQuery()` 만으로 예외가 난다. 그게 "DB에 가기 전에 잡았다"의 가장 깔끔한 증거다.

```
@Test
void 오타는_DB에_가기_전에_잡힌다() {
    assertThatThrownBy(() ->
        em.createQuery("SELECT r FROM Reservation r WHERE r.theme.nmae = :name")
    ).isNotNull();
    // 예외 타입은 직접 확인해서 채운다. 그게 실습 3의 3번 항목이다
}
```

환경이 다 됐다고 판단하는 기준

아무 JPQL이나 하나 실행했을 때, 로그에 SQM 덤프와 SQL AST 덤프와 최종 SQL이 순서대로 찍힌다. 그리고 상속 전략을 `TABLE_PER_CLASS` 로 바꿔도 부팅이 된다. 이 둘이 되면 넘어간다.

3. 실습

작업 순서

앞의 다섯 개가 발표를 만들고, 뒤의 네 개는 질문에 대비한다.

순서	실습	왜 이 순서인가
1	실습 1 (프론트엔드 수렴)	발표 2파트의 전제. 이게 없으면 SQM 이야기를 시작할 수 없다
2	실습 2 (백엔드 분기)	발표의 첫 1분이다. 없으면 발표를 시작할 수 없다
3	실습 4 (상속 3종)	킬링 포인트. 발표의 대표 이미지가 여기서 나온다
4	실습 3 (의미 분석)	3파트 2분을 지탱하는 유일한 직접 증거
5	실습 5 (두 갈래 lowering)	파이프라인 그림을 정확하게 그리기 위해
6	실습 6 (TYPE 연산자)	실습 4와 세팅을 공유해서 거의 공짜
7	실습 7 (N+1 정량)	Q&A 백업
8	실습 8 (플랜 캐시)	Q&A 백업
9	실습 9 (ANTLR 문법)	Q&A 백업

실습 1. 세 갈래 입력이 정말 한 곳으로 모이는가

발표 2파트에서 "쿼리를 만드는 방법은 셋인데 결국 같은 곳으로 간다"고 말한다. 그 말이 맞는지 확인한다.

먼저 그림을 고쳐야 한다

이 실습에서 제일 중요한 건 코드를 돌리는 게 아니라 그림을 바로잡는 것이다.

HQL, Criteria, Spring Data 메서드 이름을 나란한 세 갈래로 그리면 안 된다.

`PartTreeJpaQuery` 는 메서드 이름을 쪼갬 다음 `JpaQueryCreator` 를 통해 `CriteriaQuery` 를 조립한다. 즉 메서드 이름 파싱은 `Criteria`와 나란한 형제가 아니라 **Criteria 위에 얹혀 있는 층**이다.

셋을 평행하게 그린 슬라이드는 "Spring Data가 Criteria를 거치는데 왜 옆에 나란히 그렸냐"는 질문 하나에 흔들린다.

계단으로 그리면 오히려 그림이 세진다.

```

findByStoreId(...) 메서드 이름
    | PartTreeJpaQuery -> JpaQueryCreator
    v
Criteria API <----- @Query("SELECT r FROM ...")
    |                               | HqlParser + SemanticQueryBuilder
    +-----+-----+
                v
                SQM
  
```

프론트엔드 위에 프론트엔드가 쌓이고 그 아래에 IR 하나가 있는 구조가 된다. 여기에 "Criteria 트리가 곧 SQM이다"라는 사실을 없으면 이런 문장이 나온다.

"메서드 이름이 SQM 노드가 되기까지, 문자열은 한 번도 등장하지 않습니다."

평행 구조로는 나올 수 없는 문장이다.

준비

같은 의미의 쿼리를 세 가지 방법으로 작성한다.

```
// 경로 A: HQL 문자열
em.createQuery("SELECT r FROM Reservation r WHERE r.store.id = :storeId",
               Reservation.class)
    .setParameter("storeId", 1L);

// 경로 B: Criteria API
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Reservation> cq = cb.createQuery(Reservation.class);
Root<Reservation> r = cq.from(Reservation.class);
cq.select(r).where(cb.equal(r.get("store").get("id"), 1L));

// 경로 C: Spring Data 메서드 이름
List<Reservation> findByStoreId(Long storeId);
```

절차

1. 세 경로의 SQM 덤프를 로그에서 캡처해 나란히 놓고 비교한다.
2. 로그만 믿지 말고 SQM 객체를 코드로 직접 꺼내본다. Query 를 SqmQueryImplementor 로 unwrap하면 SqmStatement 를 얻을 수 있다.
3. [탐색] Criteria가 SQM으로 "변환되는" 게 아니라 애초에 SQM이라는 걸 소스에서 확인한다. 볼 곳은 세 군데다. SqmCriteriaNodeBuilder 가 하이버네이트의 CriteriaBuilder 구현체인지, SqmFrom 이 JPA의 From 을 직접 구현하는지, SqmSelectStatement 가 CriteriaQuery 를 구현하는지.
4. [탐색] 경로 C가 어느 지점에서 합류하는지 스택 트레이스로 확인한다. PartTreeJpaQuery 부터 타고 들어간다.

4번은 반드시 본인 버전에서 확인한다. **Spring Data 버전에 따라 Criteria를 조립하는 경로를 탈 수도 있고, HQL 문자열을 만들어서 파서를 태우는 경로를 탈 수도 있다.** 후자라면 위에 그린 계단의 방향이 반대가 되므로 슬라이드를 다르게 그려야 한다.

성공 기준

세 경로의 SQM 노드 구조가 일치한다. 경로 C의 합류 지점을 스택 트레이스로 짚을 수 있다. 3번의 세 클래스를 IDE에서 열어 캡처해뒀다.

합정

세 덤프가 완전히 똑같이 나오지는 않을 가능성이 크다. 파라미터를 표현하는 방식이나 alias를 붙이는 방식에서 차이가 난다. 이때 **그 차이가 의미의 차이인지 표기의 차이인지 판단하는 것이 이 실습의 진짜 난이도다.** 판단 기준을 먼저 세우고 적용한다.

여유가 되면

Spring Data는 `@Query` 문자열에 정렬을 덧붙이거나 count 쿼리를 파생시키려고 자체 ANTLR JPQL 문법을 따로 들고 있다. 파서가 두 개인 셈이다. 재미있는 사실이지만 본문에는 넣지 않는다.

슬라이드로 뽑을 것

세 덤프를 나란히 놓은 슬라이드 한 장. 계단 구조로 그린다.

실습 2. DB를 바꾸면 SQL이 어디서부터 갈라지는가

이 실습의 결과물이 발표의 첫 1분 전체다. 여기서 못 뽑으면 발표를 시작할 수 없다.

도입 혹은 이렇게 돌아간다. JPQL 한 줄을 보여주고, 그게 만들어낸 SQL 두 개를 보여준다. 자바 코드는 한 글자도 안 바뀌었는데 SQL의 마지막 줄이 다르다. 문자열 치환이라면 일어날 수 없는 일이니까, 중간에 뭔가가 있다는 결론으로 넘어간다.

절차

같은 JPQL을 MySQL과 PostgreSQL Dialect로 각각 실행하고 SQL을 비교한다. 볼 만한 지점이 네 군데 있는데 우선순위가 다르다.

관측 대상	roomescape 쿼리	볼 것	용도
페이징	예약 목록 조회 + <code>setMaxResults(10)</code>	<code>limit</code> 대 <code>offset fetch first</code>	도입 혹. 필수
문자열 연결	<code>concat(t.name, ' - ', s.name)</code>	함수 형태인가 연산자 형태인가	4-3 보강
날짜 비교	<code>WHERE r.date >= :from AND r.date < :to</code>	날짜 리터럴 표현	예비
원도 함수	대기 순번 <code>row_number() over (...)</code>	지원 여부와 표현 차이	체감용

페이징은 반드시 확보한다. 나머지는 여유가 있으면 찍는다. 혹 슬라이드에는 차이가 나는 마지막 한 줄만 색을 달리해서 올린다.

원도 함수를 굳이 넣는 이유가 있다. 이미 `ReservationWaitDao` 에서 `ROW_NUMBER() OVER (PARTITION BY ...)` 를 손으로 쓰고 있다. 같은 걸 HQL로 표현했을 때 Dialect마다 어떻게 내려가는지 보면, "내가 손으로 쓰던 SQL이 IR을 거쳐 다시 만들어진다"는 감각이 생긴다.

갈라지는 지점 찾기

발표 4-3에서 도입부 미스터리를 회수한다. 그때 필요한 게 이 부분이다.

1. [탐색] `SqlAstTranslator` 인터페이스의 구현체 목록을 IDE에서 연다.
`MySQLSqlAstTranslator`, `PostgreSQLSqlAstTranslator`, `OracleSqlAstTranslator` 같은 것들이 줄줄이 나온다. 이 목록을 캡처한 스크린샷 한 장이 4-3 슬라이드다.
2. [탐색] `Dialect` 가 개입할 수 있는 지점을 확인한다. `getHqlTranslator`, `getSqmTranslatorFactory`, `getSqlAstTranslatorFactory` 셋인데, 실제 `Dialect` 구현들이 주로 어디에 집중하는지 본다.
3. SQM 덤프는 같은지, SQL AST 덤프까지 같은지, 아니면 SQL AST부터 갈라지는지 확인한다.

3번을 반드시 한다. 대본은 "SQL AST까지도 사실상 같았고 갈라진 건 마지막 한 걸음"이라고 말하는데, 덤프로 확인하지 않았으면 그 문장을 쓰면 안 된다. 갈라지는 지점이 더 앞이면 문장을 고쳐야 한다.

성공 기준

SQM 덤프는 같은데 최종 SQL만 다른 스냅샷을 확보했다. 그리고 어느 단계에서 갈라지는지 덤프로 짚을 수 있다.

슬라이드로 뽑을 것

도입 혹은 슬라이드 한 장(JPQL 한 줄 + SQL 두 개), 4-3 슬라이드 한 장(`SqlAstTranslator` 구현체 목록). 실습 1과 합쳐 모래시계 슬라이드 한 장.

실습 3. 하이버네이트가 DB에 가기 전에 오류를 잡는가

발표 3파트는 "SQM에는 구문이 아니라 의미가 들어 있다"는 주장에 2분을 쓴다. 그런데 지금까지 이 주장의 근거는 `SqmExpression<T>` 가 제네릭을 갖는다는 소스 사실 하나뿐이었다. 청중이 화면에서 볼 수 있는 증거가 없었다.

이 실습은 비용이 거의 안 드는데 그 증거를 만들어준다.

절차

```
// 문법은 완벽하다. 점 찍고 식별자를 썼을 뿐이다.
em.createQuery("SELECT r FROM Reservation r WHERE r.theme.nmae = :name");
```

1. `org.hibernate.SQL` 로거를 켜 상태로 이 쿼리를 실행한다.
2. **DB 로그가 비어 있는 화면을 캡처한다.** 예외 스택 트레이스와 텅 빈 SQL 로그가 한 화면에 들어가야 한다. 이게 슬라이드다.
3. [탐색] 어느 클래스가 이 예외를 던지는지 확인한다. 그 클래스 이름이 이 실습의 결론이다.
4. [탐색] 세 프론트엔드가 각각 언제 이 오류를 잡는지 확인한다. 경로 A는 쿼리를 만들 때인가 실행할 때인가. 경로 B는 `r.get("name")` 를 호출하는 순간인가. 경로 C는 애플리케이션이 뜰 때인가 호출할 때인가.
5. [탐색] `@Query` 문자열은 언제 검증되는지 확인한다. 기동 시점에 검사하는 옵션이 있는지 본다.
6. [탐색] 디버거로 `r.theme.name` 의 SQM 노드를 열어서, `SqmExpression<T>` 의 타입 파라미터가 실제로 `String` 으로 잡혀 있는지 본다.

주장의 수위 조절

발표 3-2에서 "세 프론트엔드가 각각 다른 시점에 잡습니다"라고 말하려면 4번과 5번을 다 확인한 뒤에 말한다. **확인하지 않았으면 2번까지만 보여주고 넘어간다.** 화면 하나만으로도 충분히 강하다.

성공 기준

예외가 어느 클래스에서 던져지는지 스택 트레이스로 짚을 수 있다. 세 프론트엔드의 검출 시점을 표로 정리했다.

이렇게 나오면 오히려 좋다

4번의 답이 세 경로에서 다르게 나오는 경우다. 같은 오류를 서로 다른 시점에 잡는다는 건 프론트엔드가 정말 셋이라는 부수적 증거가 된다. 다만 발표에서는 한 문장으로만 언급하고 넘어간다.

실습 4. 매핑 전략만 바꿨을 때 SQL이 얼마나 달라지는가

발표 5파트 전체가 이 실습에서 나온다. 킬링 포인트다.

이 실습을 발표에 넣는 이유

한때 이걸 발표에서 빼려고 했었다. 이유는 타당했다. 청중이 `JOINED` 와 `TABLE_PER_CLASS` 의 차이를 모르면, 슬라이드를 띄우는 순간 발표가 lowering 설명에서 상속 매핑 설명으로 갈아

타버린다는 것이었다.

그런데 지금 대본의 5파트는 **상속 매핑을 설명하지 않는다**. 필요한 건 전략별 사용법이나 트레이드오프가 아니라 "같은 질문인데 SQL의 모양 자체가 달라진다"는 인식 하나뿐이다.

구체적으로 이렇게 다르다.

	빠려고 했던 버전	지금 버전
슬라이드에 올리는 것	세 전략의 SQL 전문	세 전략의 SQL 한 줄씩
청중에게 요구하는 것	전략별 구조를 이해할 것	"모양이 다르네"라고 느낄 것
발표자가 하는 말	언제 무엇을 쓰는가	@Inheritance 한 줄만 바꿨습니다
필요한 사전 지식	있음	없음

청중 조건도 유리하다. 방탈출 미션을 한 사람은 전원이 예약과 대기를 한 테이블로 합칠지 나눌지 고민해봤다. 상속 매핑 전략의 이름은 몰라도 그 설계 질문은 안다. 발표는 그 질문에서 출발한다.

안전장치: 리허설에서 5파트가 1분 30초를 넘거나 청중이 헤매는 기색이면 실습 6(TYPE 연산자)으로 대체한다. 그쪽은 슬라이드 반 장에 30초라 예산이 남는다. 그래서 실습 6을 버리지 않고 예비로 남겨둔다.

세팅

실습 0에서 SEQUENCE 로 세팅해줬다면 바로 진행한다.

```

@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE) // 여기만 바꿔가며 관측
abstract class SlotClaim {
    @Id @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY) private Member member;
    @ManyToOne(fetch = FetchType.LAZY) private Theme theme;
    @ManyToOne(fetch = FetchType.LAZY) private ReservationTime time;
    @ManyToOne(fetch = FetchType.LAZY) private Store store;
    private LocalDate date;
}

@Entity class Reservation extends SlotClaim { }
@Entity class Waiting extends SlotClaim {
    private LocalDateTime createdAt;
}

```

```
SELECT c FROM SlotClaim c WHERE c.member.id = :memberId
```

이 한 줄이 미션 2단계 요구사항("예약과 대기가 상태로 구분되어 함께 표시된다")과 정확히 대응한다. 지금 구현은 두 테이블을 따로 조회해서 자바에서 합치고 있다.

관측

@Inheritance 전략을 바꿔가며 SQL을 찍는다.

전략	나올 SQL	확인할 것
SINGLE_TABLE	단일 테이블 + 판별 컬럼 조건	판별 컬럼 비교로 끝나는가
JOINED	부모와 자식 테이블들의 LEFT JOIN	판별을 무엇으로 대신하는가
TABLE_PER_CLASS	자식 테이블들의 UNION ALL	UNION 서브쿼리에 붙는 상수 컬럼의 정체는

반드시 확인할 것

전략을 바꿨을 때 SQM 덤프가 동일한가.

발표 5파트 전체가 "SQM에서 이건 노드 하나입니다"라는 문장 위에 서 있다. 전략에 따라 SQM이 달라진다면 그 문장이 틀린 것이고, 킬링 포인트를 통째로 다시 써야 한다.

세 조건 각각에서 SQM 덤프를 캡처해서 diff를 뜬다. 완전히 같지 않다면 어디가 다른지, 그게 의미의 차이인지 확인한다. 이 검증은 건너뛴 수 없다.

성공 기준

세 SQM 덤프가 일치하고 세 SQL의 모양이 다르다. 이 대비를 한 화면에 담았다.

슬라이드로 뿜을 것

킬링 포인트 슬라이드 한 장. 위에 SQM 덤프 하나, 아래에 세 SQL을 한 줄씩. 발표에서 SQL 세 개를 다 읽지 않고 짚기만 한다.

Q&A 총알

TABLE_PER_CLASS 의 UNION에 등장하는 상수 컬럼(clazz_ 같은 것)의 정체를 설명할 수 있게 준비해둔다. 런타임에 타입을 알아내야 하는 상황이므로 컴파일러의 어떤 기법과 같은 구조인지까지 답할 수 있으면 좋다.

실습 5. 번역 결과물이 정말 두 개인가

발표 6파트에서 "SQM을 번역하면 결과물이 두 개 나온다"고 말한다. 하나는 DB로 보낼 SQL AST고, 다른 하나는 돌아온 행을 객체로 조립하는 계획인 DomainResult 그래프다.

앞의 것만 있으면 쿼리는 나가지만 결과를 엔티티로 만들 수가 없다. 컴파일러로 치면 기계어와 별개로 필요한 런타임 메타데이터에 해당한다.

절차

1. 실습 0의 로깅 설정에 `org.hibernate.orm.results.graph.AST` 가 DEBUG로 들어가 있는지 확인한다. 덤프가 안 나오면 그 절의 주의사항 두 가지를 먼저 점검한다.
2. 실습 1의 경로 A 쿼리를 실행하고, SQL AST 덤프와 DomainResult 덤프를 같은 로그에서 캡처한다.
3. JOIN FETCH 를 붙인 쿼리로 같은 걸 캡처한다. roomescape에서는 `SELECT r FROM Reservation r JOIN FETCH r.theme`.
4. 두 덤프를 비교한다. JOIN FETCH 가 무엇을 바꿨는가. SQL AST만 바뀌었는가, DomainResult 그래프도 바뀌었는가, 둘 다인가.

5. [탐색] `SqmTranslator` 의 반환 타입을 소스에서 확인한다. 무엇을 담고 있는가.

성공 기준

`SqmTranslator` 가 SQL AST 하나만 돌려주는 게 아니라 조립 계획을 함께 돌려준다는 것을 **반환 타입으로** 짚을 수 있다.

합정

DomainResult 덤프는 출력량이 많다. 엔티티 하나만 조회하는 쿼리로 시작하고, 연관이 붙은 쿼리는 그다음에 본다.

슬라이드로 뽑을 것

새 슬라이드를 만들지 않는다. **기존 파이프라인 그림을 고친다.** 마지막 단계를 하나가 아니라 두 갈래로 그린다.

실습 6. 없는 개념을 어떻게 흉내내는가 (TYPE 연산자)

여기서부터는 슬라이드로 가지 않는 실습이다.

`TYPE()` 은 "소스 언어에는 있는데 타겟 언어에는 없는 개념"의 가장 순수한 예다. SQL에는 "이 행의 자바 타입"이라는 개념이 아예 없다. 없는 개념을 있는 재료로 흉내내는 과정을 SQL 한 줄로 보여줄 수 있다.

실습 4와 세팅을 공유하므로 추가 비용이 거의 없다. `SINGLE_TABLE` 하나만 있으면 된다.

```
SELECT c FROM SlotClaim c WHERE c.member.id = :memberId AND TYPE(c) = Waiting
```

`TYPE(c)` 가 SQL의 어느 조각으로 바뀌는지 확인하고, 화살표 하나로 그릴 수 있으면 성공이다.

쓸 데가 두 군데다. 실습 4가 리허설에서 시간을 넘기면 5파트를 이걸로 대체한다(반 장, 30 초). 대체하지 않으면 Q&A 총알로 쏜다.

실습 7. N+1은 실제로 몇 개인가

발표에서 N+1에 쓰는 시간은 **40초**다. 이 실습의 결과물은 Q&A 백업 슬라이드 한 장이고, 질문이 나올 때만 꺼낸다.

앞에서 한 번 경고했지만 다시 적는다. 이 표가 잘 나오면 본문에 넣고 싶어진다. 넣는 순간 발표가 N+1 발표로 되돌아간다.

7-A. 계측기부터 교정한다

이 실습을 할 거면 이 단계를 건너뛰면 안 된다. `Statistics`의 카운터들이 서로 다른 것을 세기 때문이다.

가장 위험한 오해는 이거다. `getQueryExecutionCount()`는 **HQL, Criteria, 네이티브 쿼리의 실행 횟수만 센다**. 지연 로딩 프록시가 초기화되면서 나가는 secondary select는 여기 안 잡힌다. 예약 128건을 순회하며 프록시를 128번 건드려도 이 카운터는 1을 뺀다. 이 카운터로 N+1의 N을 재면 N은 영원히 0이다.

교정 절차는 이렇다.

1. 예약 목록을 JPQL로 조회한다(쿼리 1회).
2. 결과 중 하나의 `theme`을 건드려 프록시를 초기화한다(secondary select 1회).
3. 네 카운터를 모두 찍는다.

```
Statistics stats = sessionFactory.getStatistics();
stats.getQueryExecutionCount();      // HQL/Criteria/native 실행 수
stats.getPrepareStatementCount();    // JDBC PreparedStatement 준비 수
stats.getEntityFetchCount();         // 지연 로딩으로 추가 조회된 엔티티 수
stats.getEntityLoadCount();          // 로딩된 엔티티 총 수
```

4. `datasource-proxy`가 잡은 실제 쿼리 수와 대조한다.

여기서 무엇을 쓸지 확정한다. N+1의 N은 `getEntityFetchCount()`로 센다(의미가 가장 직접적이다). 총 왕복 수는 `getPrepareStatementCount()`로 센다. 교차 검증은 `datasource-proxy`로 한다. `getQueryExecutionCount()`는 실습 1과 8에서만 쓴다.

7-B. 측정

대상은 `SELECT r FROM Reservation r WHERE r.store.id = :storeId`를 실행한 뒤 `theme`에 접근하는 상황이다.

조건	예상 쿼리 수	볼 것
LAZY, 전체 순회	1 + (서로 다른 Theme 수)	N의 정의를 정확히 잡는다
EAGER, JPQL 조회	1 + (서로 다른 Theme 수)	매핑 정보가 있어도 쓰이지 않는다
JOIN FETCH	1	힌트가 적용된다
@EntityGraph(attributePaths = {"theme"})	1	다른 문법, 같은 효과
@BatchSize(size = S)	1 + $\lceil$ (서로 다른 Theme 수) / S $\rceil$	차선책으로 내려앉는다

roomescape의 data.sql 기준 분포는 Store 3개, Theme 3에서 11개, ReservationTime 11개, Member 8개, Reservation 128건 이상이다.

데이터 세팅은 두 가지로 돌린다. 세팅 A는 지금 data.sql 그대로다(Theme이 적어서 참조가 공유된다). 세팅 B는 예약 건마다 서로 다른 Theme을 붙인다(공유 없음). **두 세팅의 쿼리 수가 왜 달라지는지 설명할 수 있어야 한다.** 이 설명이 곧 영속성 컨텍스트를 이해했다는 증거다.

EAGER 조건은 주의한다. `find()` 는 EAGER 연관을 LEFT JOIN으로 가져오지만 JPQL로 조회하면 secondary select를 쓴다. **버전에 따라 다르므로 반드시 본인 버전에서 찍은 로그로 확인한다.** 재현이 안 되면 Q&A에서 "버전에 따라 다릅니다"로 받는다.

측정할 때 빠지기 쉬운 함정

첫째, JVM 워밍업. 첫 실행은 JIT가 안 돌아서 비정상적으로 느리다. 몇 번 돌린 뒤 측정한다.

둘째, 영속성 컨텍스트 초기화. 측정 사이에 `clear()` 를 호출하지 않으면 1차 캐시 때문에 두 번째 측정부터 쿼리가 안 나간다. roomescape처럼 참조 대상이 적은 도메인에서는 이 함정에 걸리면 측정이 통째로 의미가 없어진다.

셋째, 시간이 아니라 쿼리 수가 주장의 축이다. 로컬 컨테이너 DB는 네트워크 지연이 없어서 N+1의 실제 비용이 드러나지 않는다. 시간은 참고치로만 제시한다.

넷째, 통계 초기화. `Statistics` 는 `SessionFactory` 단위로 누적된다. 조건마다 `stats.clear()` 를 부르거나 델타를 계산한다.

실습 8. 파싱을 매번 하지는 않는가 (쿼리 플랜 캐시)

발표 아웃트로가 30초로 줄면서 캐시 이야기는 본문에서 빠졌다. Q&A 총알로만 남는다.

기본 측정

```
// A: 값을 문자열에 직접 박는다. 매번 캐시 미스
for (long id = 1; id <= 1000; id++) {
    em.createQuery("SELECT r FROM Reservation r WHERE r.member.id = " + id)
        .getResultList();
}

// B: 파라미터 바인딩. 첫 회만 미스
for (long id = 1; id <= 1000; id++) {
    em.createQuery("SELECT r FROM Reservation r WHERE r.member.id = :id")
        .setParameter("id", id).getResultList();
}
```

```
stats.getQueryPlanCacheHitCount();
stats.getQueryPlanCacheMissCount();
```

캐시 오염

```
spring.jpa.properties.hibernate.query.plan_cache_max_size=16
```

캐시 크기를 줄이고 서로 다른 쿼리 문자열 100개를 날려서 히트율이 떨어지는 양상을 본다.

Criteria도 캐시되는가

실습 1에서 세 프론트엔드가 같은 SQM으로 모이는 걸 봤다면 자연스럽게 따라오는 질문이다. Criteria로 만든 쿼리도 플랜 캐시에 들어가는가.

경로 B를 1000회 반복하고 히트 카운트를 본다. 결과가 "조건부"이거나 "아니오"에 가까우면, "Criteria는 문자열 파싱을 건너뛰니까 더 효율적"이라는 통념과 정면으로 부딪힌다. 좋은 Q&A 총알이 된다.

총을 섞지 않는다

총	캐싱 대상	위치
하이버네이트 쿼리 플랜 캐시	컴파일 결과	JVM 힙
JDBC 드라이버 statement 캐시	PreparedStatement 객체	커넥션

층	캐싱 대상	위치
DB 실행 계획 캐시	실행 계획	DB 서버

parameter sniffing은 세 번째 층의 문제다. 첫 번째 층을 설명하다가 여기로 미끄러지지 않게 조심한다.

실습 9. 문법 파일 확인

하이버네이트 레포의 `hibernate-core/src/main/antlr/` 아래에서 `HqlParser.g4` 와 `HqlLexer.g4` 를 열어 캡처 한 장을 확보한다. 발표 본문에는 넣지 않는다.

여유가 있으면 파스 트리를 시각화해본다.

```
antlr4 -Dlanguage=Java HqlParser.g4
grun HqlParser statement -gui
```

4. 과제

실습 1과 2를 마친 뒤

Q1. [탐색] [사고] 메서드 이름이 SQM에 도달하기까지 거치는 클래스를 순서대로 나열하라. 스택 트레이스를 근거로 댈 것. 그리고 이 경로를 HQL, Criteria와 나란히 그리면 안 되는 이유를 한 문장으로 정리하라.

Q2. [사고] 세 경로의 SQM이 완전히 같지 않았다면, 차이가 난 지점을 나열하고 각각이 의미의 차이인지 표기의 차이인지 판정하라. 판정 기준을 먼저 세우고 적용할 것.

Q3. [탐색] [사고] Hibernate 5까지 Criteria 쿼리는 HQL 문자열로 렌더링된 다음 다시 파싱됐다. 6에서 이 왕복이 사라졌다. 이 변화가 "IR을 두면 무엇이 좋은가"에 대한 증거가 되는 이유를 설명하라. 그리고 왜 이게 이론적 논증($M \times N$ 대 $M + N$)보다 강한 근거인지 답하라.

Q4. [탐색] [사고] 실습 2에서 SQM과 SQL AST 중 어디까지 같았는가. "갈라지는 건 마지막 한 걸음"이라는 대본 문장이 관측과 일치하는가. 일치하지 않으면 문장을 어떻게 고쳐야 하는가.

Q5. [탐색] SQM은 같은데 SQL만 다르다는 걸 확인했다면, "Dialect가 백엔드다"라는 주장에 대한 반례를 하나 만들어보라. 확인 경로는 `Dialect` 클래스의 `public` 메서드 목록 전체

를 훑는 것이다.

실습 3과 4를 마친 뒤

Q6. [탐색] [사고] 오타 낸 필드명이 세 프론트엔드에서 각각 언제 잡히는가. 검출 시점이 다른 이유를 설명하라. 그리고 이 차이가 "프론트엔드가 셋"이라는 주장에 어떤 근거를 더하는지 답하라.

Q7. [탐색] [사고] @Inheritance 전략만 바꿨을 때 SQM 덤프가 같았는가. 같지 않았다면 어디가 달랐고, 그것이 "SQM에서 이건 노드 하나입니다"라는 문장을 무효로 만드는가. 만든다면 대본을 어떻게 고쳐야 하는가.

Q8. [탐색] [사고] TABLE_PER_CLASS 의 UNION에 상수 컬럼이 필요한 이유를 설명하고, 이것이 컴파일러의 어떤 기법과 같은 구조인지 답하라. 힌트는 런타임에 타입을 알아내야 하는 상황이라는 점이다.

Q9. [사고] 세 전략의 SQL을 다 본 뒤, 지금 roomescape가 reservation 과 reservation_wait 을 별개 테이블로 유지하는 선택이 상속 전략 중 무엇에 해당하는지 답하라. 그리고 그 선택이 치르고 있는 비용과 얻고 있는 이득을 하나씩 쓰라. 이 답은 미션 회고 용이지 슬라이드용이 아니다.

Q10. [탐색] JOIN FETCH 의 재조립을 "컴파일 시점에 만들어진 조립 계획"이라고 고쳐 말한다면, 그 계획은 SQM에 있어야 하는가, SQL AST에 있어야 하는가, 제3의 자리인가. 실습 5의 덤프와 SqmTranslator 의 반환 타입으로 답하라.

나머지 실습을 마친 뒤

Q11. [탐색] [사고] 실습 7-A의 네 카운터가 각각 무엇을 세는지 한 문장씩 쓰고, LAZY 조건에서 왜 값이 갈라지는지 설명하라. 그리고 getQueryExecutionCount() 가 지연 로딩 select 를 세지 않는 것이 버그가 아니라 설계인 이유를 답하라. 힌트는 이 카운터 이름에 있는 "query"가 무엇을 가리키느냐다.

Q12. [내부] [사고] 예약 128건을 순회하며 theme에 접근했을 때 나간 쿼리 수를 적고, 그 수가 나온 이유를 설명하라. "1차 캐시 때문"이라는 한 줄 답은 인정하지 않는다. 프록시가 언제 생성되고 어디에 등록되고 무엇을 키로 조회되는지를 순서대로 서술할 것.

Q13. [사고] 세팅 B에서는 쿼리가 몇 개 나갔는가. 그리고 이 결과가 "N+1"이라는 이름 자체에 던지는 문제는 무엇인가.

Q14. [탐색] [사고] fetch = EAGER 로 바꿔도 N+1이 발생하는 이유를 컴파일러 용어로 설명하라. "정보가 없다"는 쓸 수 없다. EAGER는 정보가 있는 상황이다.

Q15. [사고] @BatchSize 를 컴파일러 최적화 중 무엇에 대응시킬 것인가. 대본은 그냥 "힌트"라고만 했는데 더 정확한 대응물이 있다. 힌트는 @BatchSize 가 아직 요청되지 않은 것까

지 미리 가져온다는 점이다.

Q16. [탐색] [사고] 실습 8의 Criteria 캐싱 측정 결과를 근거로, "Criteria가 HQL보다 효율적이다"라는 주장의 성립 조건을 다시 쓰라. 무조건 참도 무조건 거짓도 아니다.

5. 더 생각해볼 것

SQM은 왜 AST가 아니라 Semantic Model인가 [탐색]

이건 이제 발표의 본체다. 3파트 2분이 전부 여기에 걸려 있다.

`SqmExpression<T>`가 제네릭 타입 파라미터를 갖는다는 점을 확인한다. AST는 구문 구조만 담지만 SQM은 타입 정보를 노드에 박아서 들고 다닌다. 의미 분석의 결과가 자료구조에 굳어 있는 것이다.

여기에 실습 3의 결과를 붙이면 "의미가 들어 있다"는 주장이 소스 근거와 실행 근거를 둘 다 갖게 된다.

IR의 정의 문제 [사고]

대본은 "SQM은 IR이다"라고 단언하지 않고 "high-level IR에 해당하는 역할을 한다"고 말한다. 예상 질문 A-0이 정확히 이 표현을 겨냥한다.

답할 것은 셋이다. IR에 형식적인 정의가 있는가, 없다면 통상 어떤 성질들로 이야기되는가. 그 성질들 중 SQM이 갖는 것과 갖지 않는 것은 무엇인가. "프론트엔드와 백엔드를 분리해 준다"가 왜 IR의 효용이지 충분조건이 아닌가.

이 과제의 결과물은 슬라이드가 아니라 **Q&A용 30초 답변 스크립트**다. 외워둔다.

하이버네이트가 하지 않는 최적화 [사고]

폐기해야 할 문장이 하나 있다. "타깃이 또 하나의 옵티마이저인 컴파일러는 흔치 않다."

사실이 아니다. C를 타깃으로 삼는 트랜스파일러는 전부 최종 최적화를 gcc나 clang에 넘긴다. 이 문장을 그대로 발표하면 "Nim도 그런데요" 한 마디에 흔들린다.

대신 이렇게 잡는다. 특이한 건 타깃이 옵티마이저라는 사실이 아니라, **그 옵티마이저의 비용 모델이 컴파일러 바깥의 런타임 통계에 의존한다는 점**이다. gcc의 판단 기준은 적어도 소스 안에 있다. DB 옵티마이저의 판단 기준은 테이블 통계와 카디널리티 추정치인데, 그건 하이버네이트가 볼 수도 없고 영향을 줄 수도 없다.

이렇게 고치면 원래 결론이 오히려 세진다. 하이버네이트는 "무엇을 가져올지"만 정하고 "어떻게 가져올지"는 넘길 수밖에 없다. 최적화 책임이 두 층으로 나뉘면 어느 층도 책임지지 않는 틈이 생기고, N+1이 정확히 그 틈에 있다.

답할 것: 그 틈에 있는 다른 문제를 하나 더 찾아라. N+1 말고.

주장이 좁아진 과정 [사고]

주장을 두 번 좁혔다.

1. "ORM은 컴파일러다"
2. "하이버네이트는 컴파일러와 런타임을 모두 갖춘 언어 구현체다"
3. "하이버네이트 안에 컴파일러가 있다"

각 단계에서 무엇을 포기하고 무엇을 얻었는지 설명하라. 그리고 왜 3번이 2번보다 나은지 답하라. 힌트는 3번이 오늘 발표에서 증명할 수 있는 것만 주장한다는 점이다.

참고로 언어 구현체에 대응시키면 이렇게 된다.

하이버네이트	언어 구현체 대응
JPQL 처리 경로 (SQM, SQL AST)	컴파일러
DomainResult 그래프	런타임 메타데이터
영속성 컨텍스트	관리되는 힙
더티 체크	쓰기 감시
flush 순서 결정	지연 쓰기, 커밋 시점 반영
쿼리 플랜 캐시	JIT 코드 캐시

6. 미션 회고로 넘길 것

실습 4의 세팅과 과제 Q9는 미션의 슬롯 도메인을 다시 설계하기 위한 사전 조사를 겸한다. 발표가 끝나면 이 결과를 회고 문서로 따로 정리한다.

지금 roomescape는 reservation 과 reservation_wait 을 별개 테이블로 두고, 대기 목록을 조회할 때 자바에서 합치고 있다. 세 전략의 SQL을 본 뒤에 이 선택을 다시 평가한다. 슬라이드에는 올리지 않는다.

7. 실습 결과에 따라 대본을 고칠 곳

- ☐ 실습 2 결과에 따라 4-3의 "갈라지는 건 마지막 한 걸음" 문장을 검증하거나 수정
- ☐ 실습 3 결과에 따라 3-2의 "세 프론트엔드가 각각 다른 시점에 잡는다" 문장을 추가하거나 삭제
- ☐ 실습 4의 SQM diff 결과에 따라 5파트의 "노드 하나입니다" 문장을 검증
- ☐ 실습 5 결과로 파이프라인 그림을 두 갈래로 수정
- ☐ Q3(Hibernate 5의 왕복 제거) 답이 나오면 3-3에 20초 분량으로 넣을지 검토
- ☐ 실습 7에서 EAGER N+1이 재현되는지에 따라 Q&A C-1의 단언 수위 조정
- ☐ 리허설에서 5파트가 1분 30초를 넘으면 실습 6(TYPE)으로 대체

8. 참고 자료

소스에서 직접 확인할 곳

목적	위치
HQL 문법	hibernate-core/src/main/antlr/HqlParser.g4 , HqlLexer.g4
파싱 결과로 SQM 만들기	SemanticQueryBuilder
Criteria가 곧 SQM 이라는 근거	SqmCriteriaNodeBuilder , SqmFrom , SqmSelectStatement
SQM 노드	org.hibernate.query.sqm.tree
SQM에서 SQL AST 로 내리기	StandardSqmTranslator
SQL AST 노드	org.hibernate.sql.ast.tree
방언별 SQL 렌더링	SqlAstTranslator 인터페이스와 구현체 목록
조립 계획	org.hibernate.sql.results.graph (DomainResult, Fetch)
덤프 로거	DomainResultGraphPrinter
백엔드 진입점	org.hibernate.dialect.Dialect
카운터 의미	org.hibernate.stat.Statistics
메서드 이름 파싱	org.springframework.data.jpa.repository.query.PartTreeJpaQuery
Criteria 조립 지점	org.springframework.data.jpa.repository.query.JpaQueryCreator

공식 문서

- Hibernate 6 User Guide
- Hibernate ORM 6.0 릴리스 페이지 (HQL과 Criteria가 같은 SQM으로 모인다는 공식 서술)
- Hibernate ORM 6.0 Alpha3 릴리스 노트 (Criteria 트리가 곧 SQM이라는 서술, Q3의 근거)
- Spring Boot Reference
- MySQL 8.4 Reference Manual

블로그 대신 javadoc과 릴리스 노트를 1차 자료로 쓴다. 발표에서 단언하는 문장은 전부 이 문서의 실습 중 하나에 대응되어야 한다.

도서

- 토비의 스프링 vol.1, 6장 AOP. "프록시가 언제 무엇을 대신하는가"를 다시 읽으면 지연 로딩 프록시가 선명해진다.
- 오브젝트(조영호). 상속 매핑 전략 3종을 "타입 계층을 데이터로 표현하는 세 가지 방법"으로 읽으면 실습 4에 직접 도움이 된다.
- SQL 첫걸음(아사이 아츠시). UNION과 서브쿼리 구조 복습이 필요하면.

9. 리허설 체크리스트

- ☐ 모래시계 슬라이드에서 프론트엔드를 계단으로 그렸는가 (평행 아님)
- ☐ 도입 혹은 두 SQL을 실제 MySQL과 PostgreSQL에서 뽑아 왔는가 (H2 아님)
- ☐ 도입 1분 동안 N+1이라는 단어를 한 번도 말하지 않았는가
- ☐ 3-3에서 "IR의 정의입니다"가 아니라 "해당하는 역할을 합니다"라고 말했는가
- ☐ 4-3에서 도입부 미스터리를 확실히 회수했는가
- ☐ 5파트에서 SQL 세 개를 다 읽지 않았는가 (한 줄씩만)
- ☐ 5파트가 1분 30초를 넘지 않았는가
- ☐ 상속 실습의 SQM diff를 확인한 뒤에 "노드 하나입니다"라고 말했는가
- ☐ N+1 파트가 40초 안에 끝났는가
- ☐ N+1 파트에서 "오늘 주제가 아니니 여기까지"를 말했는가
- ☐ N+1 측정 표를 본문에 띄우지 않았는가
- ☐ 아웃트로 네 문장을 암기했는가
- ☐ ALL(*)를 본문에서 자세히 설명하지 않았는가
- ☐ Q&A 총알이 정리되어 있는가 (IR의 정의, ALL(*), SSA/CFG 부재, parameter sniffing, EAGER N+1, DomainResult, Spring Data 자체 파서, Criteria 캐싱, TYPE 연산자,

TABLE_PER_CLASS 상수 컬럼)

☐ 전체가 9분 30초 안에 끝났는가

부록. 이전 실습 문서에서 바뀐 것

실습 내용 자체가 틀려서 고친 게 아니다. 대본의 중심축이 N+1에서 SQM으로 옮겨가면서 무엇이 슬라이드로 직행하는지가 바뀐 것이다.

실습	이전	지금	이유
백엔드 분기	발표용 실습 중 하나	도입 1분 전체	도입 혹은 EAGER N+1에서 Dialect 분기로 교체됐다
상속 3종	부록(발표 제외)	킬링 포인트	5파트가 상속 매핑을 설명하지 않게 되면서 사전 지식 부담이 사라졌다
의미 분석	없음	신설	"SQM에 의미가 들어 있다"를 말로만 주장하고 있었다
TYPE 연산자	발표용	상속 실습의 대타	상속이 발표로 올라가면서 자리를 넘겼다
N+1 정량	발표 1분 30초	Q&A 백업	발표 시간이 40초로 줄었다
계측기 교정	필수 선행 단계	N+1 실습에 통합	N+1을 안 하면 교정도 필요 없다
플랜 캐시	Outro 근거	Q&A 백업	아웃트로가 30초로 줄면서 캐시 언급이 빠졌다

세팅에서 한 가지가 바뀌었다. 이전에는 `SEQUENCE` 매핑을 상속 실습에 한해서만 쓰기로 했는데, 상속이 발표용으로 올라가면서 **처음부터 전 테이블에 적용**해야 한다.

전체 작업량은 오히려 줄었다. 발표용 실습이 다섯 개로 늘었지만, N+1 5종 측정과 캐시 A/B 측정이라는 무거운 계측 실습 두 개가 백업으로 내려갔고 계측기 교정이라는 선행 조건도 함께 빠졌기 때문이다.